

Submission Worksheet

Submission Data

Course: IT114-003-F2025

Assignment: IT114 Milestone 2 - Hangman

Student: Alanor S. (ass89)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 12/8/2025 10:53:54 AM

Updated: 12/8/2025 10:48:22 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-2-hangman/grading/ass89>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-2-hangman/view/ass89>

Instructions

1. Refer to Milestone2 of [Hangman / Word guess](#)
 1. Complete the features
 2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
 3. Switch to the Milestone2 branch
 1. `git checkout Milestone2`
 2. `git pull origin Milestone2`
 4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
 5. Once finished, click "Submit and Export"
 6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. ``git commit -m "adding PDF"`
 3. `git push origin Milestone2`
 4. On Github merge the pull request from Milestone2 to main
 7. Upload the same PDF to Canvas
 8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`
- Complete each section and task sequentially.
 - Review the details and validation criteria for each task.
 - Ensure subtasks are completed before the parent task.

Section #1: (1 pt.) Payloads

Progress: 100%

≡ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Reqs from the document
 - Provided Payload for applicable items that only need client id, message, and type
 - PointsPayload for syncing points of players
 - Each payload will be presented by debug output (i.e, properly override the toString() method like the lesson examples)

Part 1:

Progress: 100%

Details:

- Show the code related to your payloads (Payload, PointsPayload, and any new ones added)
- Each payload should have an overridden toString() method showing its internal data

Connection Payload from Common

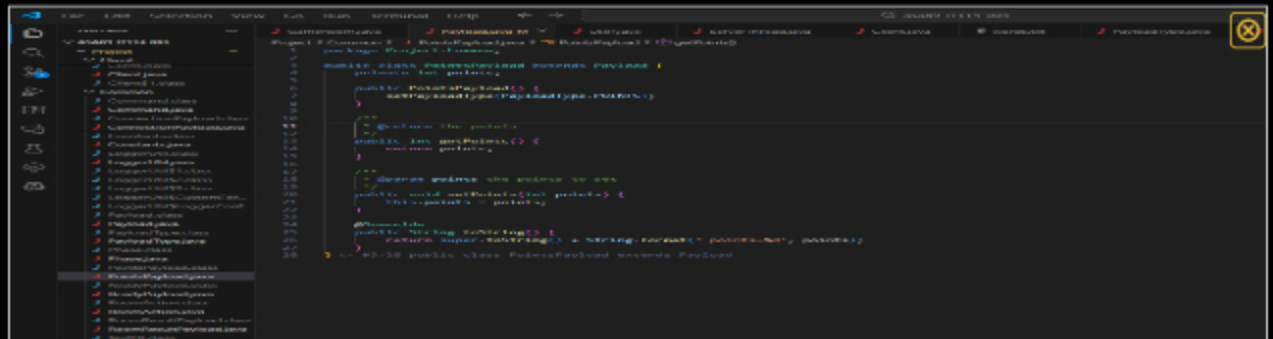
Payload from Common PT1

Payload from Common PT2

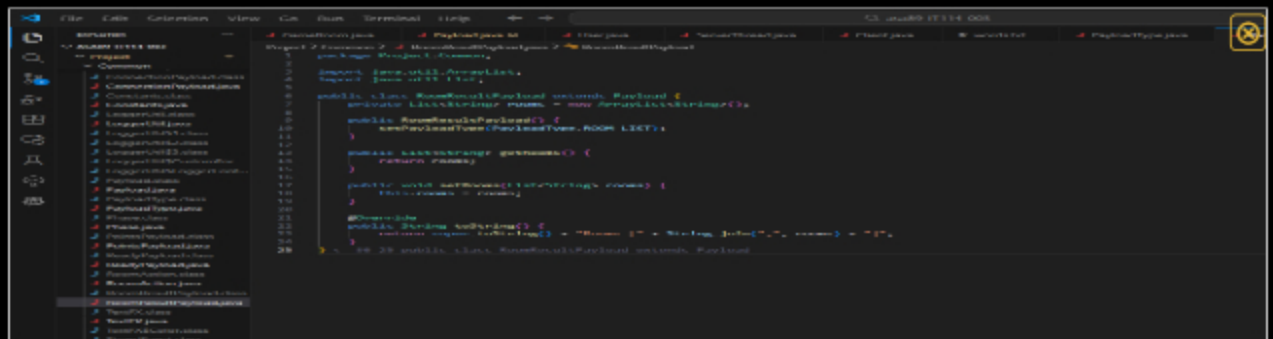




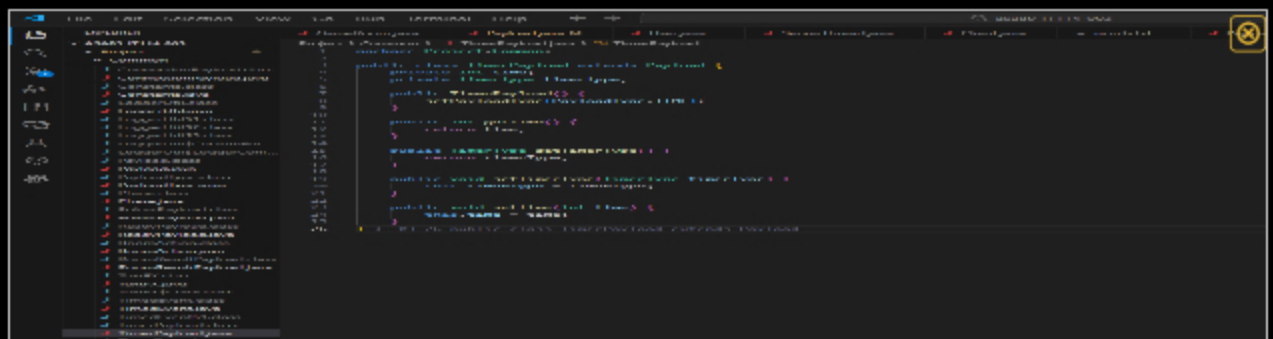
PayloadType from Common



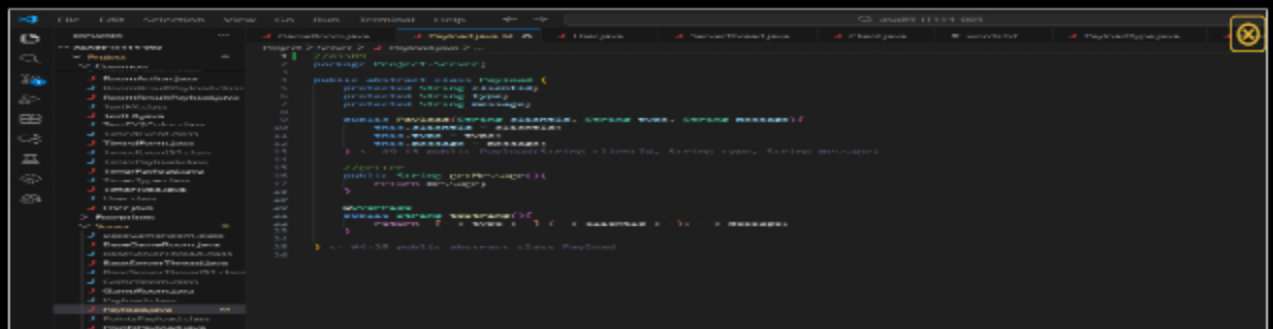
PointsPayload from Common



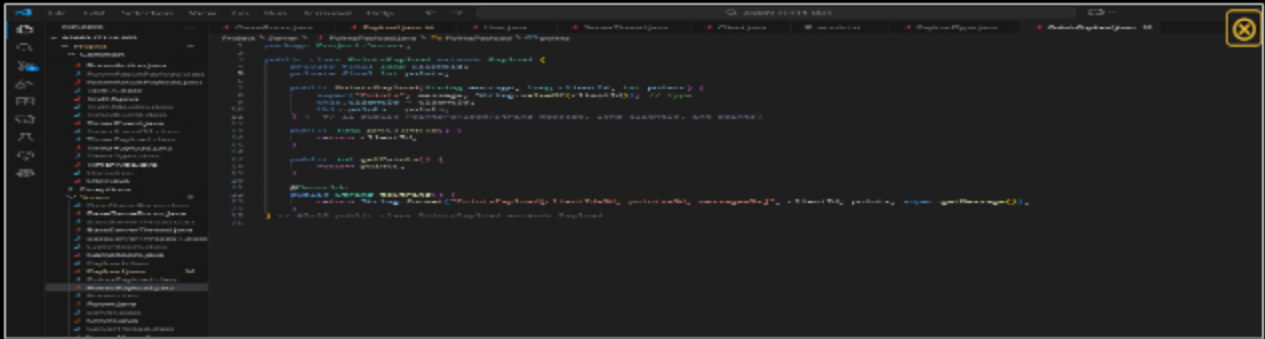
RoomResults Payload from Common



TimerPayload from Common



Payload from Server



PointsPayload from Server



Saved: 12/8/2025 8:07:43 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

Your Response:

Payload.java in Common folder:

- base class for most payloads sent between client and server. It has 3 core pieces of data:
1. **payloadType**: tells the receiver what kind of message this is (like for example MESSAGE, READY, POINTS etc)
 2. **clientId**: identifies which client sent the payload
 3. **message**: a string field, basically a foundation that all other specialized payloads extend

ConnectionPayload in Common:

- it extends Payload and adds one more property → **clientName**
- clientName** is the username the client provides when connecting. It's used to identify the client before they receive an official client ID

PointPayload in Common:

- specialized payload that represents changes in the player's score
- points**: the number of points the client earned, which is sent from the server to clients to sync scores during the game

ReadyPayload in Common:

- indicates whether a client is ready to start the game
- isReady**: a boolean showing if the user entered the ready command; used by the server to coordinate when all players have signaled readiness and the game can begin

RoomResultPayload in Common:

- represents a list of rooms available on the server
- rooms: a list of room names (List); it's sent to clients when they request the room list

TimerPayload in Common:

- handles countdowns or timed phases in the game
- time: an integer representing the time
- timeType: an enum specifying what timer is running; synchronizes all clients so they get the same timing events

Payload in Server:

- has its own abstract Payload class
- clientId: the ID of the client as a string
- type: the type of the payload (used internally by the server)
- message: the main content of the payload; this server-side version is used for local handling and logging of messages

PointsPayload in Server:

- represents a sompleted scoring event
- clientId: which client has earned points
- points: how many points they recieved; used for logging and internal server communication before it's translated into the Common version of the payload sent to clients

 Saved: 12/8/2025 8:07:43 PM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

≡ Task #1 (0.57 pts.) - GameRoom Client Add/Remove

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the onClientAdded() code
- Show the onClientRemoved() code



```

1  // onClientAdded()
2  void GameRoom::onClientAdded(const Client* client) {
3      // Add client to the list of clients
4      clients.push_back(client);
5      // Update the room list
6      updateRoomList();
7      // Update the timer
8      updateTimer();
9  }
10
11 // onClientRemoved()
12 void GameRoom::onClientRemoved(const Client* client) {
13     // Remove client from the list of clients
14     clients.remove(client);
15     // Update the room list
16     updateRoomList();
17     // Update the timer
18     updateTimer();
19 }
  
```

Show the onClientAdded() code Show the onClientRemoved() code



Saved: 12/8/2025 8:22:29 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in onClientAdded() (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in onClientRemoved() (at least should handle logic for an empty session)

Your Response:

When a new client joins the game room, the onClientAdded() method makes sure that the client's local state is synchronized with the current session. Specifically, it updates the joining client with the current phase of the game, the ready status of all players, and the turn status. This makes sure that the new client sees the correct game state immediately upon joining, without affecting the turn order or changing the other players' data.

When a client leaves the game room, the onClientRemoved() method manages several key tasks to maintain session integrity. It removes the departing player from the turn order and checks whether the room is now empty. If the room is empty, it resets timers for rounds, turns, and ready status, and calls onSessionEnd() to cleanly end the session. If the player who has left was the current player in the turn order, the method automatically advances to the next player to continue gameplay.



Saved: 12/8/2025 8:22:29 PM

≡ Task #2 (0.57 pts.) - GameRoom Session Start

Progress: 100%

Details:

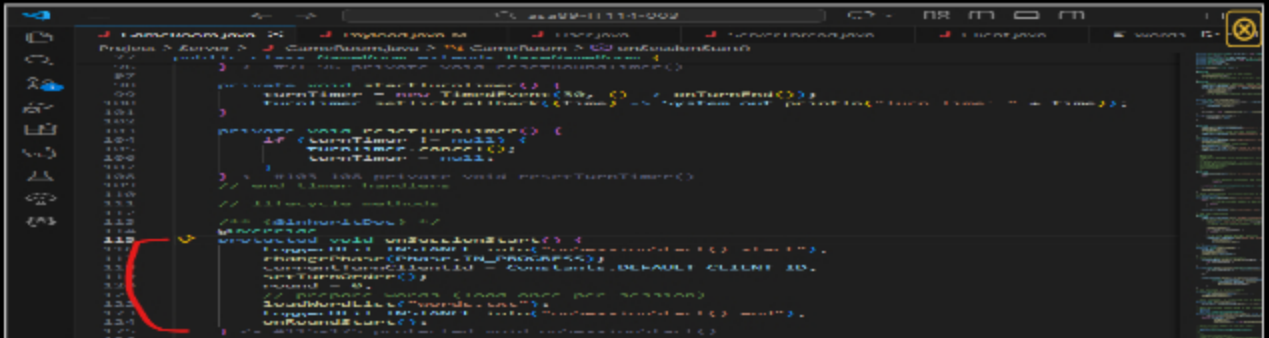
- Reqs from document
 - GameRoom loads the word list into memory from a text file for later use
 - First word is randomly chosen from in-memory list
 - Blanks are shared with all players
 - First round is triggered
- Reset/set initial state

🖼 Part 1:

Progress: 100%

Details:

- Show the snippet of onSessionStart()



```
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

Show the snippet of onSessionStart()



Saved: 12/8/2025 8:31:13 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

When a session begins, the game transitions to the IN_PROGRESS phase, signaling that gameplay can start. The turn order is started by collecting all ready players and shuffling them to create a randomized order of play. The round counter is reset to zero, and the word list for game is loaded from words.txt. After this setup, the session immediately triggers the next lifecycle event by calling onRoundStart(), which begins the first round of the game, and initializes round-specific state (like picking a new word, clearing suggested letters, resetting strikes etc). It makes sure that the game session is fully prepared and transitions smoothly to active gameplay.



Saved: 12/8/2025 8:31:13 PM

Task #3 (0.57 pts.) - GameRoom Round Start

Progress: 100%

Details:

- Reqs from Document
 - GameRoom round timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of onRoundStart()

```
13.5 // onRoundStart() - Called when a new round starts
13.6 // Parameters: None
13.7 // Returns: None
13.8 // Description: Resets the round timer and the turn status for all clients, making sure that there are no leftover
13.9 // state from the previous round carries over. The round counter is incremented, and a new word is
13.10 // selected from the word list while the display blanks are initialized to underscore characters. The
13.11 // previously guessed letters are cleared, and the global strike count is reset to zero, creating a clean
13.12 // slate for the round. The server then notifies all clients of the new round and the current blanked
13.13 // word display, making sure that everyone sees the correct state. OnTurnStart() is then called to
13.14 // determine which player begins, starting the turn-based cycle where each player can guess a letter,
13.15 // attempt to guess the full word, or skip their turn.
13.16
13.17 // Implementation
13.18 void onRoundStart() {
13.19     // Reset the round timer
13.20     roundTimer = 0;
13.21     // Increment the round counter
13.22     roundCounter++;
13.23     // Select a new word from the word list
13.24     word = wordList[roundCounter % wordList.size()];
13.25     // Initialize the display blanks to underscore characters
13.26     displayBlanks = word.length();
13.27     // Clear the previously guessed letters
13.28     guessedLetters = "";
13.29     // Reset the global strike count to zero
13.30     globalStrikeCount = 0;
13.31     // Notify all clients of the new round and the current blanked word display
13.32     for (int i = 0; i < clients.size(); i++) {
13.33         clients[i].sendRoundStart(word, displayBlanks, guessedLetters, globalStrikeCount);
13.34     }
13.35     // Call OnTurnStart() to determine which player begins
13.36     OnTurnStart();
13.37 }
```

Show the snippet of onRoundStart()

Saved: 12/8/2025 8:44:18 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

onRoundStart() prepares both the game state and the players for a new turn-based round. First, it resets the round timer and the turn status for all clients, making sure that there are no leftover state from the previous round carries over. The round counter is incremented, and a new word is selected from the word list while the display blanks are initialized to underscore characters. The previously guessed letters are cleared, and the global strike count is reset to zero, creating a clean slate for the round. The server then notifies all clients of the new round and the current blanked word display, making sure that everyone sees the correct state. OnTurnStart() is then called to determine which player begins, starting the turn-based cycle where each player can guess a letter, attempt to guess the full word, or skip their turn.

Saved: 12/8/2025 8:44:18 PM

Task #4 (0.57 pts.) - GameRoom Turn Start

Progress: 100%

Details:

- Reqs from Document
 - Pick next Player (Initially random, then round-robin)
 - Reset turn related status to allow player to do actions
 - GameRoom Turn timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of onTurnStart()

```

181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Show the snippet of onTurnStart()

Saved: 12/8/2025 9:05:03 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

When a round begins, `setTurnOrder()` collects all ready players and shuffles them to establish a randomized turn order. The `currentTurnClientId` tracks whose turn it is, and `getNextPlayer()` advances this ID in round-robin style. At the start of each turn, `onTurnStart()` resets the turn timer, identifies the current player, and notifies all clients that is that player's turn. Player actions are then processed, after which the player is marked as having taken their turn and other clients are updated via `sendTurnStatus()`. `onTurnEnd()` either advances to the next player or ends the round if all players have taken their turn.

Saved: 12/8/2025 9:05:03 PM

Task #5 (0.57 pts.) - GameRoom Turn End

Progress: 100%

Details:

- Reqs from Document

- **Condition 1:** Turn ends when Player guesses a letter or word (and word isn't complete)
- **Condition 2:** Turn ends when the Turn Timer expires
- **Condition 3:** Turn ends when the current Player Skips their turn
- If not the last player, trigger turn start logic
- If the last player, trigger round end logic
- If end condition is met, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of onTurnEnd()

```

117 public void onTurnEnd() {
118     // Resets logic, restores Turn Start and Turn End is typically handled via client
119     // (Another XXXXX)
120     // ...
121     // ...
122     // ...
123     // ...
124     // ...
125     // ...
126     // ...
127     // ...
128     // ...
129     // ...
130     // ...
131     // ...
132     // ...
133     // ...
134     // ...
135     // ...
136     // ...
137     // ...
138     // ...
139     // ...
140     // ...
141     // ...
142     // ...
143     // ...
144     // ...
145     // ...
146     // ...
147     // ...
148     // ...
149     // ...
150     // ...
151     // ...
152     // ...
153     // ...
154     // ...
155     // ...
156     // ...
157     // ...
158     // ...
159     // ...
160     // ...
161     // ...
162     // ...
163     // ...
164     // ...
165     // ...
166     // ...
167     // ...
168     // ...
169     // ...
170     // ...
171     // ...
172     // ...
173     // ...
174     // ...
175     // ...
176     // ...
177     // ...
178     // ...
179     // ...
180     // ...
181     // ...
182     // ...
183     // ...
184     // ...
185     // ...
186     // ...
187     // ...
188     // ...
189     // ...
190     // ...
191     // ...
192     // ...
193     // ...
194     // ...
195     // ...
196     // ...
197     // ...
198     // ...
199     // ...
200     // ...
201     // ...
202     // ...
203     // ...
204     // ...
205     // ...
206     // ...
207     // ...
208     // ...
209     // ...
210     // ...
211     // ...
212     // ...
213     // ...
214     // ...
215     // ...
216     // ...
217     // ...
218     // ...
219     // ...
220     // ...
221     // ...
222     // ...
223     // ...
224     // ...
225     // ...
226     // ...
227     // ...
228     // ...
229     // ...
230     // ...
231     // ...
232     // ...
233     // ...
234     // ...
235     // ...
236     // ...
237     // ...
238     // ...
239     // ...
240     // ...
241     // ...
242     // ...
243     // ...
244     // ...
245     // ...
246     // ...
247     // ...
248     // ...
249     // ...
250     // ...
251     // ...
252     // ...
253     // ...
254     // ...
255     // ...
256     // ...
257     // ...
258     // ...
259     // ...
260     // ...
261     // ...
262     // ...
263     // ...
264     // ...
265     // ...
266     // ...
267     // ...
268     // ...
269     // ...
270     // ...
271     // ...
272     // ...
273     // ...
274     // ...
275     // ...
276     // ...
277     // ...
278     // ...
279     // ...
280     // ...
281     // ...
282     // ...
283     // ...
284     // ...
285     // ...
286     // ...
287     // ...
288     // ...
289     // ...
290     // ...
291     // ...
292     // ...
293     // ...
294     // ...
295     // ...
296     // ...
297     // ...
298     // ...
299     // ...
300     // ...
301     // ...
302     // ...
303     // ...
304     // ...
305     // ...
306     // ...
307     // ...
308     // ...
309     // ...
310     // ...
311     // ...
312     // ...
313     // ...
314     // ...
315     // ...
316     // ...
317     // ...
318     // ...
319     // ...
320     // ...
321     // ...
322     // ...
323     // ...
324     // ...
325     // ...
326     // ...
327     // ...
328     // ...
329     // ...
330     // ...
331     // ...
332     // ...
333     // ...
334     // ...
335     // ...
336     // ...
337     // ...
338     // ...
339     // ...
340     // ...
341     // ...
342     // ...
343     // ...
344     // ...
345     // ...
346     // ...
347     // ...
348     // ...
349     // ...
350     // ...
351     // ...
352     // ...
353     // ...
354     // ...
355     // ...
356     // ...
357     // ...
358     // ...
359     // ...
360     // ...
361     // ...
362     // ...
363     // ...
364     // ...
365     // ...
366     // ...
367     // ...
368     // ...
369     // ...
370     // ...
371     // ...
372     // ...
373     // ...
374     // ...
375     // ...
376     // ...
377     // ...
378     // ...
379     // ...
380     // ...
381     // ...
382     // ...
383     // ...
384     // ...
385     // ...
386     // ...
387     // ...
388     // ...
389     // ...
390     // ...
391     // ...
392     // ...
393     // ...
394     // ...
395     // ...
396     // ...
397     // ...
398     // ...
399     // ...
400     // ...
401     // ...
402     // ...
403     // ...
404     // ...
405     // ...
406     // ...
407     // ...
408     // ...
409     // ...
410     // ...
411     // ...
412     // ...
413     // ...
414     // ...
415     // ...
416     // ...
417     // ...
418     // ...
419     // ...
420     // ...
421     // ...
422     // ...
423     // ...
424     // ...
425     // ...
426     // ...
427     // ...
428     // ...
429     // ...
430     // ...
431     // ...
432     // ...
433     // ...
434     // ...
435     // ...
436     // ...
437     // ...
438     // ...
439     // ...
440     // ...
441     // ...
442     // ...
443     // ...
444     // ...
445     // ...
446     // ...
447     // ...
448     // ...
449     // ...
450     // ...
451     // ...
452     // ...
453     // ...
454     // ...
455     // ...
456     // ...
457     // ...
458     // ...
459     // ...
460     // ...
461     // ...
462     // ...
463     // ...
464     // ...
465     // ...
466     // ...
467     // ...
468     // ...
469     // ...
470     // ...
471     // ...
472     // ...
473     // ...
474     // ...
475     // ...
476     // ...
477     // ...
478     // ...
479     // ...
480     // ...
481     // ...
482     // ...
483     // ...
484     // ...
485     // ...
486     // ...
487     // ...
488     // ...
489     // ...
490     // ...
491     // ...
492     // ...
493     // ...
494     // ...
495     // ...
496     // ...
497     // ...
498     // ...
499     // ...
500     // ...
501     // ...
502     // ...
503     // ...
504     // ...
505     // ...
506     // ...
507     // ...
508     // ...
509     // ...
510     // ...
511     // ...
512     // ...
513     // ...
514     // ...
515     // ...
516     // ...
517     // ...
518     // ...
519     // ...
520     // ...
521     // ...
522     // ...
523     // ...
524     // ...
525     // ...
526     // ...
527     // ...
528     // ...
529     // ...
530     // ...
531     // ...
532     // ...
533     // ...
534     // ...
535     // ...
536     // ...
537     // ...
538     // ...
539     // ...
540     // ...
541     // ...
542     // ...
543     // ...
544     // ...
545     // ...
546     // ...
547     // ...
548     // ...
549     // ...
550     // ...
551     // ...
552     // ...
553     // ...
554     // ...
555     // ...
556     // ...
557     // ...
558     // ...
559     // ...
560     // ...
561     // ...
562     // ...
563     // ...
564     // ...
565     // ...
566     // ...
567     // ...
568     // ...
569     // ...
570     // ...
571     // ...
572     // ...
573     // ...
574     // ...
575     // ...
576     // ...
577     // ...
578     // ...
579     // ...
580     // ...
581     // ...
582     // ...
583     // ...
584     // ...
585     // ...
586     // ...
587     // ...
588     // ...
589     // ...
590     // ...
591     // ...
592     // ...
593     // ...
594     // ...
595     // ...
596     // ...
597     // ...
598     // ...
599     // ...
600     // ...
601     // ...
602     // ...
603     // ...
604     // ...
605     // ...
606     // ...
607     // ...
608     // ...
609     // ...
610     // ...
611     // ...
612     // ...
613     // ...
614     // ...
615     // ...
616     // ...
617     // ...
618     // ...
619     // ...
620     // ...
621     // ...
622     // ...
623     // ...
624     // ...
625     // ...
626     // ...
627     // ...
628     // ...
629     // ...
630     // ...
631     // ...
632     // ...
633     // ...
634     // ...
635     // ...
636     // ...
637     // ...
638     // ...
639     // ...
640     // ...
641     // ...
642     // ...
643     // ...
644     // ...
645     // ...
646     // ...
647     // ...
648     // ...
649     // ...
650     // ...
651     // ...
652     // ...
653     // ...
654     // ...
655     // ...
656     // ...
657     // ...
658     // ...
659     // ...
660     // ...
661     // ...
662     // ...
663     // ...
664     // ...
665     // ...
666     // ...
667     // ...
668     // ...
669     // ...
670     // ...
671     // ...
672     // ...
673     // ...
674     // ...
675     // ...
676     // ...
677     // ...
678     // ...
679     // ...
680     // ...
681     // ...
682     // ...
683     // ...
684     // ...
685     // ...
686     // ...
687     // ...
688     // ...
689     // ...
690     // ...
691     // ...
692     // ...
693     // ...
694     // ...
695     // ...
696     // ...
697     // ...
698     // ...
699     // ...
700     // ...
701     // ...
702     // ...
703     // ...
704     // ...
705     // ...
706     // ...
707     // ...
708     // ...
709     // ...
710     // ...
711     // ...
712     // ...
713     // ...
714     // ...
715     // ...
716     // ...
717     // ...
718     // ...
719     // ...
720     // ...
721     // ...
722     // ...
723     // ...
724     // ...
725     // ...
726     // ...
727     // ...
728     // ...
729     // ...
730     // ...
731     // ...
732     // ...
733     // ...
734     // ...
735     // ...
736     // ...
737     // ...
738     // ...
739     // ...
740     // ...
741     // ...
742     // ...
743     // ...
744     // ...
745     // ...
746     // ...
747     // ...
748     // ...
749     // ...
750     // ...
751     // ...
752     // ...
753     // ...
754     // ...
755     // ...
756     // ...
757     // ...
758     // ...
759     // ...
760     // ...
761     // ...
762     // ...
763     // ...
764     // ...
765     // ...
766     // ...
767     // ...
768     // ...
769     // ...
770     // ...
771     // ...
772     // ...
773     // ...
774     // ...
775     // ...
776     // ...
777     // ...
778     // ...
779     // ...
780     // ...
781     // ...
782     // ...
783     // ...
784     // ...
785     // ...
786     // ...
787     // ...
788     // ...
789     // ...
790     // ...
791     // ...
792     // ...
793     // ...
794     // ...
795     // ...
796     // ...
797     // ...
798     // ...
799     // ...
800     // ...
801     // ...
802     // ...
803     // ...
804     // ...
805     // ...
806     // ...
807     // ...
808     // ...
809     // ...
810     // ...
811     // ...
812     // ...
813     // ...
814     // ...
815     // ...
816     // ...
817     // ...
818     // ...
819     // ...
820     // ...
821     // ...
822     // ...
823     // ...
824     // ...
825     // ...
826     // ...
827     // ...
828     // ...
829     // ...
830     // ...
831     // ...
832     // ...
833     // ...
834     // ...
835     // ...
836     // ...
837     // ...
838     // ...
839     // ...
840     // ...
841     // ...
842     // ...
843     // ...
844     // ...
845     // ...
846     // ...
847     // ...
848     // ...
849     // ...
850     // ...
851     // ...
852     // ...
853     // ...
854     // ...
855     // ...
856     // ...
857     // ...
858     // ...
859     // ...
860     // ...
861     // ...
862     // ...
863     // ...
864     // ...
865     // ...
866     // ...
867     // ...
868     // ...
869     // ...
870     // ...
871     // ...
872     // ...
873     // ...
874     // ...
875     // ...
876     // ...
877     // ...
878     // ...
879     // ...
880     // ...
881     // ...
882     // ...
883     // ...
884     // ...
885     // ...
886     // ...
887     // ...
888     // ...
889     // ...
890     // ...
891     // ...
892     // ...
893     // ...
894     // ...
895     // ...
896     // ...
897     // ...
898     // ...
899     // ...
900     // ...
901     // ...
902     // ...
903     // ...
904     // ...
905     // ...
906     // ...
907     // ...
908     // ...
909     // ...
910     // ...
911     // ...
912     // ...
913     // ...
914     // ...
915     // ...
916     // ...
917     // ...
918     // ...
919     // ...
920     // ...
921     // ...
922     // ...
923     // ...
924     // ...
925     // ...
926     // ...
927     // ...
928     // ...
929     // ...
930     // ...
931     // ...
932     // ...
933     // ...
934     // ...
935     // ...
936     // ...
937     // ...
938     // ...
939     // ...
940     // ...
941     // ...
942     // ...
943     // ...
944     // ...
945     // ...
946     // ...
947     // ...
948     // ...
949     // ...
950     // ...
951     // ...
952     // ...
953     // ...
954     // ...
955     // ...
956     // ...
957     // ...
958     // ...
959     // ...
960     // ...
961     // ...
962     // ...
963     // ...
964     // ...
965     // ...
966     // ...
967     // ...
968     // ...
969     // ...
970     // ...
971     // ...
972     // ...
973     // ...
974     // ...
975     // ...
976     // ...
977     // ...
978     // ...
979     // ...
980     // ...
981     // ...
982     // ...
983     // ...
984     // ...
985     // ...
986     // ...
987     // ...
988     // ...
989     // ...
990     // ...
991     // ...
992     // ...
993     // ...
994     // ...
995     // ...
996     // ...
997     // ...
998     // ...
999     // ...
1000    // ...

```

Show the snippet of onTurnEnd()

Saved: 12/8/2025 9:25:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

onTurnEnd() manages the transition between players' turns and makes sure the game flow moves on nicely. When a player's turn ends, either because they completed their action or their turn timer expired, the method first resets the turn timer to prevent any leftover countdowns. It then checks whether the current player is the last in the turn order using isLastPlayer(). If they are, onTurnEnd() triggers the end of the round by calling onRoundEnd(). Otherwise, it moves on to the next player's turn by calling onTurnStart(), which updates the currentTurnClientId and notifies all clients whose turn it is. It accurately tracks each player's actions while keeping the turn-based setup of the game. It also takes care of unusual situations, like missing or removed players, by catching errors and stopping the game from stalling.

Saved: 12/8/2025 9:25:27 PM

Task #6 (0.57 pts.) - GameRoom Round End

Progress: 100%

Details:

- Reqs from Document
 - Condition 1:** Round ends when word is completed
 - Condition 2:** Round ends when Hangman Strikes reach limit
 - Condition 3:** Round ends when the last player's turn finishes
 - If word was completed, pick new word, reset hangman, sync blanks (if not last round)
 - Send the in-progress scoreboard to all clients sorted by highest points to lowest
 - Trigger Round Start if end condition not met; otherwise, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of onRoundEnd()

```

function onRoundEnd() {
    // Check if the round should end
    if (wordCompleted || strikes === 6 || lastPlayerTurn) {
        // Round ends
        // 1. Update scoreboard
        updateScoreboard();
        // 2. Pick a new word
        pickNewWord();
        // 3. Reset hangman
        resetHangman();
        // 4. Sync blanks
        syncBlanks();
        // 5. Trigger Round Start
        triggerRoundStart();
    } else {
        // Round continues
        // 6. Trigger Next Player's Turn
        triggerNextPlayerTurn();
    }
}
    
```

Show the snippet of onRoundEnd()

Saved: 12/8/2025 9:20:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup, end checks, and next lifecycle events)

Your Response:

onRoundEnd() handles the conclusion of a single round and determines how the game should proceed based on the current state. When called, it first checks whether the round should truly

end, which happens if the word has been fully solved or if all players have reached the maximum number of strikes. If either condition is met, the method reveals the current word to all clients, resets the strike counter, and sends an updated scoreboard showing each player's points. It then checks whether the session has reached maximum number of rounds. If it does, it ends the session by calling `onSessionEnd()`. Otherwise, it starts a new round with `onRoundStart()`, which resets turn status, picks a new word, clears guessed letters, and prepares the next round. If the word is still unsolved and players have strikes remaining, the method moves on to the next player's turn by calling `onTurnStart()`.



Saved: 12/8/2025 9:20:26 PM

≡ Task #7 (0.57 pts.) - GameRoom Session End

Progress: 100%

Details:

- Reqs from Document
 - **Condition 1:** Session ends when X rounds have passed
 - Send the final scoreboard to all clients sorted by highest points to lowest (include a game over message)
 - Reset the player data for each client server-side and client-side (do not disconnect them or move them to the lobby)
 - A new ready check will be required to start a new session

📄 Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionEnd()`

```
247 protected void onSessionEnd() {
248     LoggerUtil.INSTANCE.info("onSessionEnd() start");
249     turnOrder.clear();
250     currentTurnClientid = Constants.DEFAULT_CLIENT_ID;
251     resetReadyStatus();
252     resetTurnStatus();
253     changePhase(Phase.READY);
254     LoggerUtil.INSTANCE.info("onSessionEnd() end");
255 }
256 // end lifecycle methods
```

Show the snippet of `onSessionEnd()`



Saved: 12/8/2025 9:29:35 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

`onSessionEnd()` handles the cleanup and reset of the game room once a session concludes, either because the maximum number of rounds has been reached or all players have completed their turns. It clears the turn order, resets the current turn client ID to a default value, and resets both the ready status and turn status of all clients. It also changes the game phase back to `READY`, meaning that the room is now idle and ready for a new session. Basically, it makes sure that all session-specific data is cleared and it returns the room to a clean, consistent state. This way, new sessions can start fresh, without any leftover state from the previous game. It serves as a key step for finishing the session and getting the room ready for the next lifecycle events.



Saved: 12/8/2025 9:29:35 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

≡ Task #1 (1.50 pts.) - Word Logic

Progress: 100%

Details:

- Reqs from document
 - Command: `/guess <word>`
 - GameRoom will check against the correct word
 - If it matches (exactly)
 - The guesser will receive points for each missing letter and bonus points for solving (i.e., 2x points for each empty space)
 - Points will be stored on the Player/User object
 - Sync the points value of the Player to all Clients
 - A message will be relayed that "X guessed the correct word {word} and got {points} points".
 - The round should end
 - If no matches
 - A strike will be incurred and synced to all Clients
 - A message will be relayed that "X guessed {word} and it was wrong"
 - Their turn should end

Part 1:

Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

Client side first processClientCommand Pt1

```

cmake_minimum_required(VERSION 3.10)
project(CMakeProject1)

set(CMAKE_CXX_STANDARD 11)

add_executable(CMakeProject1 main.cpp CMakeProject1.cpp)

target_link_libraries(CMakeProject1 stdc++ stdc++fs)

```

Client side first processClientCommand Pt2

[illegible]

Client side first processClientCommand Pt3 Logic is here

ServerThread next processPayload Pt1

```
File Edit Selection View Go Run Terminal Help
ASAP-0114-003
Project > Server > ServerThread.java
22 public class ServerThread extends BaseServerThread {
23     // Constructor
24     public ServerThread(Socket s, ServerSocket serverSocket, BaseServerThread bsth) {
25         super(s, serverSocket, bsth);
26     }
27     // Method to process the payload
28     public void processPayload() {
29         // Get the payload
30         byte[] payload = new byte[1024];
31         try {
32             socket.getInputStream().read(payload);
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36         // Convert the payload to a string
37         String str = new String(payload);
38         // Print the payload
39         System.out.println("Received payload: " + str);
40         // Send the payload back to the client
41         try {
42             socket.getOutputStream().write(str.getBytes());
43             socket.getOutputStream().flush();
44         } catch (IOException e) {
45             e.printStackTrace();
46         }
47     }
48 }
```

ServerThread next processPayload Pt2 CASE Guess

```
File Edit Selection View Go Run Terminal Help
ASAP-0114-003
Project > Server > ServerThread.java
22 public class ServerThread extends BaseServerThread {
23     // Constructor
24     public ServerThread(Socket s, ServerSocket serverSocket, BaseServerThread bsth) {
25         super(s, serverSocket, bsth);
26     }
27     // Method to process the payload
28     public void processPayload() {
29         // Get the payload
30         byte[] payload = new byte[1024];
31         try {
32             socket.getInputStream().read(payload);
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36         // Convert the payload to a string
37         String str = new String(payload);
38         // Print the payload
39         System.out.println("Received payload: " + str);
40         // Send the payload back to the client
41         try {
42             socket.getOutputStream().write(str.getBytes());
43             socket.getOutputStream().flush();
44         } catch (IOException e) {
45             e.printStackTrace();
46         }
47     }
48 }
```

GameRoom handling handleLetter Pt1

```
File Edit Selection View Go Run Terminal Help
ASAP-0114-003
Project > Server > GameRoom.java
22 public class GameRoom extends BaseGameRoom {
23     // Constructor
24     public GameRoom(Socket s, ServerSocket serverSocket, BaseGameRoom bsth) {
25         super(s, serverSocket, bsth);
26     }
27     // Method to handle the letter
28     public void handleLetter() {
29         // Get the letter
30         byte[] letter = new byte[1024];
31         try {
32             socket.getInputStream().read(letter);
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36         // Convert the letter to a string
37         String str = new String(letter);
38         // Print the letter
39         System.out.println("Received letter: " + str);
40         // Send the letter back to the client
41         try {
42             socket.getOutputStream().write(str.getBytes());
43             socket.getOutputStream().flush();
44         } catch (IOException e) {
45             e.printStackTrace();
46         }
47     }
48 }
```

GameRoom handling handleLetter Pt2

```
File Edit Selection View Go Run Terminal Help
ASAP-0114-003
Project > Server > GameRoom.java
22 public class GameRoom extends BaseGameRoom {
23     // Constructor
24     public GameRoom(Socket s, ServerSocket serverSocket, BaseGameRoom bsth) {
25         super(s, serverSocket, bsth);
26     }
27     // Method to handle the letter
28     public void handleLetter() {
29         // Get the letter
30         byte[] letter = new byte[1024];
31         try {
32             socket.getInputStream().read(letter);
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36         // Convert the letter to a string
37         String str = new String(letter);
38         // Print the letter
39         System.out.println("Received letter: " + str);
40         // Send the letter back to the client
41         try {
42             socket.getOutputStream().write(str.getBytes());
43             socket.getOutputStream().flush();
44         } catch (IOException e) {
45             e.printStackTrace();
46         }
47     }
48 }
```

Show the sending/syncing of the results of this command to users

```
File Edit Selection View Go Run Terminal Help
ASAP-0114-003
Project > Server > GameRoom.java
22 public class GameRoom extends BaseGameRoom {
23     // Constructor
24     public GameRoom(Socket s, ServerSocket serverSocket, BaseGameRoom bsth) {
25         super(s, serverSocket, bsth);
26     }
27     // Method to handle the letter
28     public void handleLetter() {
29         // Get the letter
30         byte[] letter = new byte[1024];
31         try {
32             socket.getInputStream().read(letter);
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36         // Convert the letter to a string
37         String str = new String(letter);
38         // Print the letter
39         System.out.println("Received letter: " + str);
40         // Send the letter back to the client
41         try {
42             socket.getOutputStream().write(str.getBytes());
43             socket.getOutputStream().flush();
44         } catch (IOException e) {
45             e.printStackTrace();
46         }
47     }
48 }
```


Details:

- Pens from document

req from document

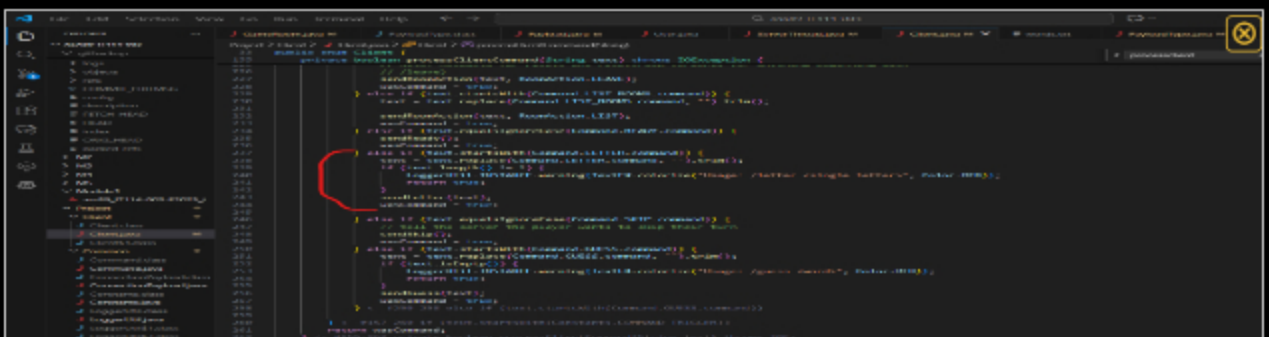
- Command: /letter <letter>
 - GameRoom will check the occurrences of the letter that haven't been guessed this round
 - If any matches
 - The guesser will receive points for each slot the letter appears in if it has not been previously guessed for this word
 - I.e., points times the number of matched letters
 - Points will be stored on the Player/User object
 - Sync the points value of the Player to all Clients
 - A message will be relayed "X guessed {letter} and there were {number} {letter}'s which yielded {points} points
 - If word is completed, round should end
 - Otherwise, just their turn should end
 - If no matches
 - A strike will be incurred and synced to all Clients
 - A message will be relayed "X guess {letter} letter, which isn't in the word"
 - If strikes reach the limit the round ends
 - If strikes don't reach the limit the current Player's turn ends

Part 1:

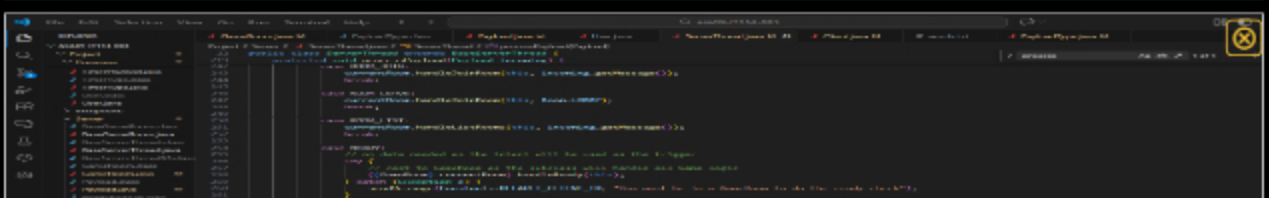
Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

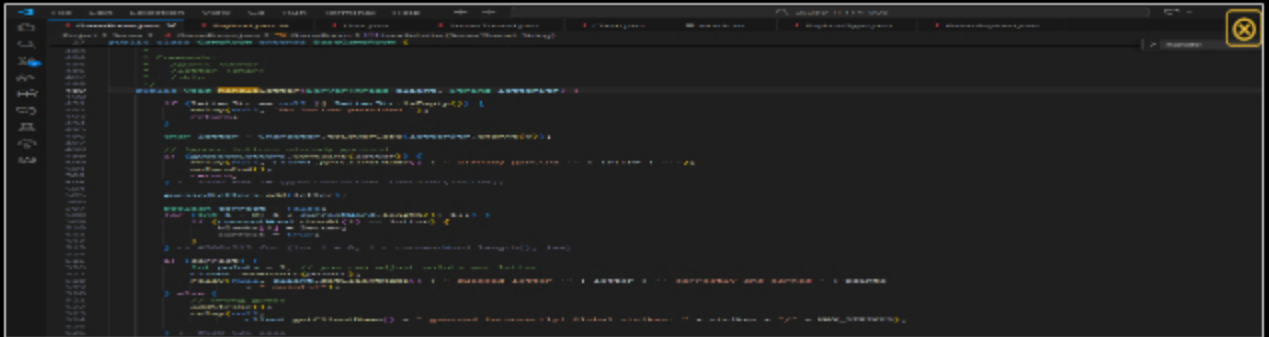


Show the Client processing of this command (process client command)

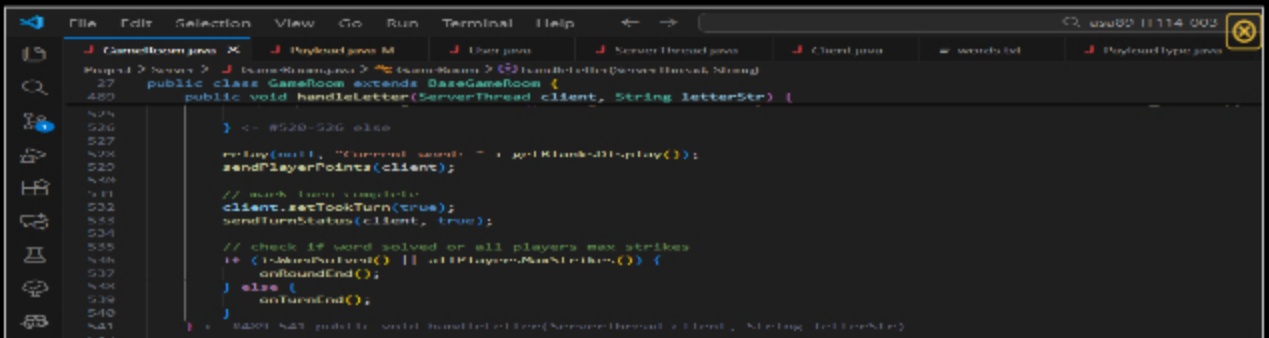




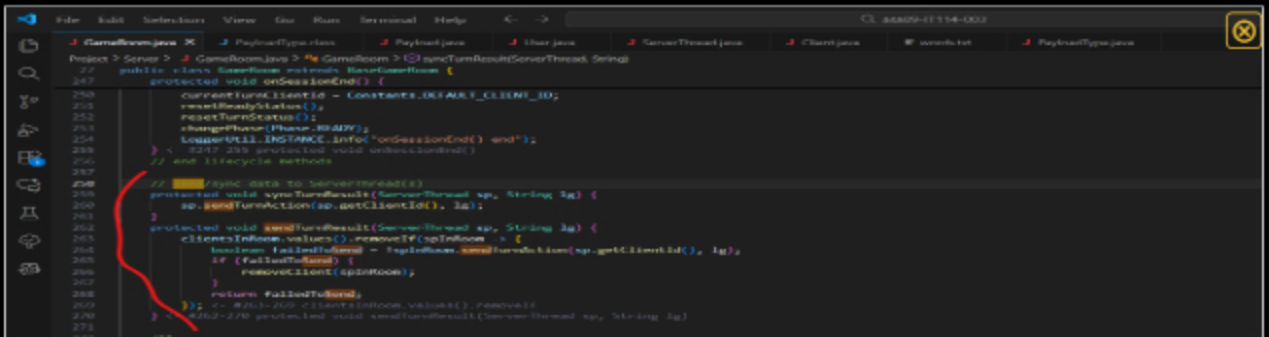
Show the ServerThread processing of this command (process method)



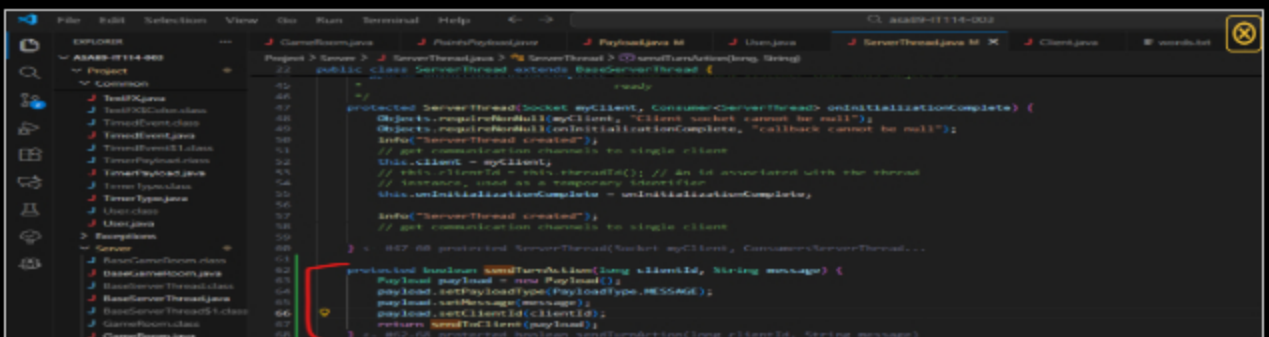
Show the GameRoom handling of this command (handle method) PT1



Show the GameRoom handling of this command (handle method) PT2



Show the sending/syncing of the results of this command to users (send/sync method) + Show the ServerThread receiving this data



Show the ServerThread receiving this data (send method)

Task #3 (0.50 pts.) - Skip Logic

Progress: 100%

Details:

- Reqs from document
 - Command: /skip
 - Skips the player's turn if it's their turn (ends their turn)

Part 1:

Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

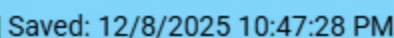
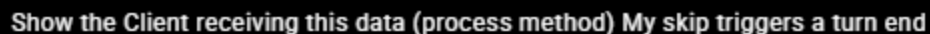
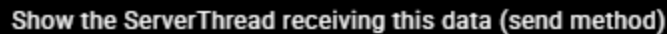
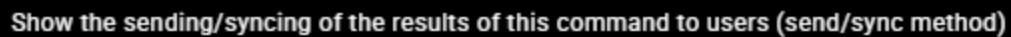
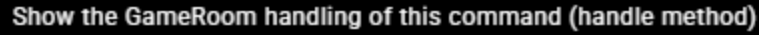
```
...  
    if (command.equals("/skip")) {  
        // Skip the player's turn if it's their turn (ends their turn)  
        GameRoom gameRoom = GameRoom.getInstance().getGameRoom(player.getId());  
        gameRoom.handleSkip(player.getId());  
    }  
    ...
```

Show the Client processing of this command (process client command)

```
...  
    if (command.equals("/skip")) {  
        // Skip the player's turn if it's their turn (ends their turn)  
        GameRoom gameRoom = GameRoom.getInstance().getGameRoom(player.getId());  
        gameRoom.handleSkip(player.getId());  
    }  
    ...
```

Show the ServerThread processing of this command (process method)

```
...  
    if (command.equals("/skip")) {  
        // Skip the player's turn if it's their turn (ends their turn)  
        GameRoom gameRoom = GameRoom.getInstance().getGameRoom(player.getId());  
        gameRoom.handleSkip(player.getId());  
    }  
    ...
```



≡ Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

When a player types `/skip` on the client, the `processClientCommand()` captures the command and calls `sendSkip()`, which sends a `SKIP` payload to the server. The `ServerThread` receives this payload in `processPayload()` and calls `handleSkip(this)` in the `GameRoom` to mark the player's turn as ended. `GameRoom` then uses the `ServerThread`'s `sendTurnStatus()` method to broadcast the turn-ending update to all connected clients. Each client receives this payload in `listenToServer()`, and `processPayload()` calls `processTurn()`, updating the player's state locally and displaying a message to show the turn has been skipped.



Saved: 12/8/2025 10:47:28 PM

Task #4 (0.50 pts.) - Game Cycle Demo

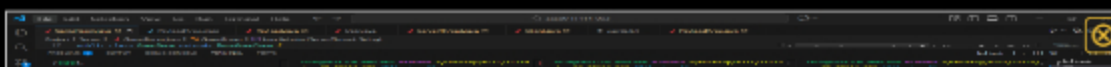
Progress: 100%

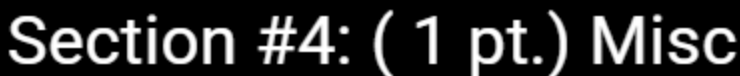
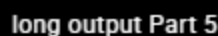
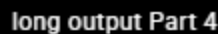
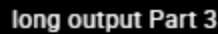
Details:

- Show examples from the terminal of a full session demonstrating each command and progress output
- This includes showing blanks, correct letters, incorrect letters, correct words, incorrect words, skips, scores, and scoreboards, etc
- Ensure at least 3 Clients and the Server are shown
- Clearly caption screenshots

long output Part 1

long output Part 2



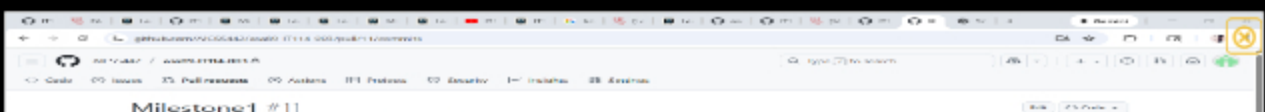


☰ Task #1 (0.33 pts.) - Github Details

Part 1:

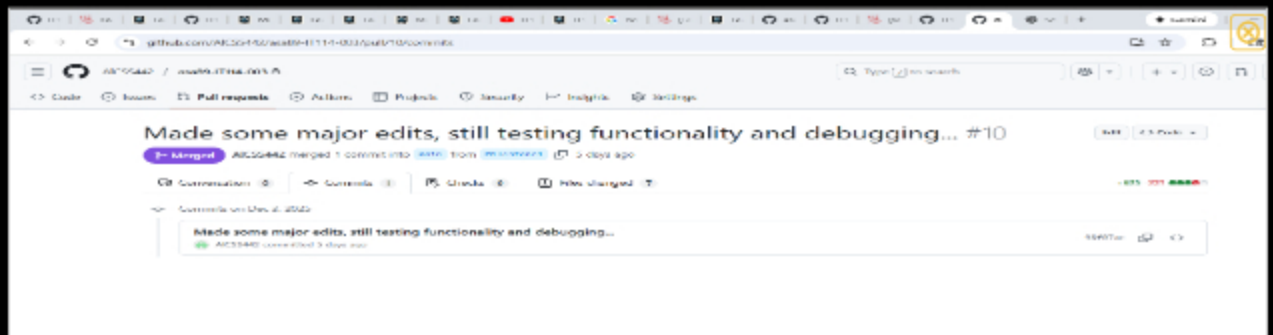
Details:

From the Commits tab of the Pull Request screenshot the commit history

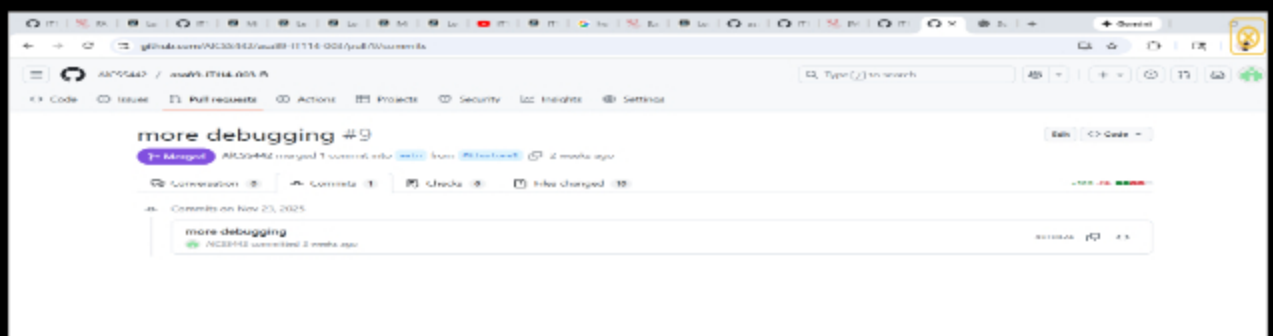




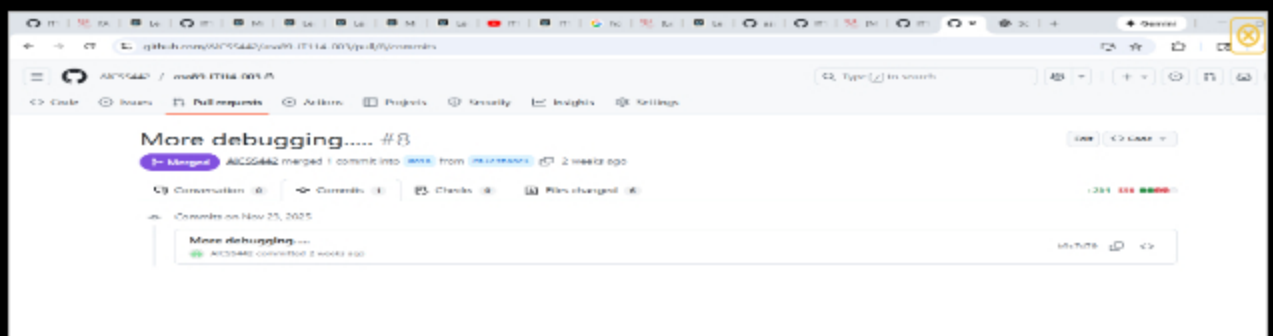
Forgot to create a branch called Milestone2, its under Milestone1



PT2



PT3



PT4



Saved: 12/8/2025 1:11:00 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#)

URL #1

<https://github.com/AICSS442/asa89-IT1140031>



URL
<https://github.com/AICSS442/asa>



URL #2

<https://github.com/AICSS442/asa89-IT1140030>



URL
<https://github.com/AICSS442/asa>



URL #3

<https://github.com/AICSS442/asa89-IT114003/>



URL
<https://github.com/AICSS442/asa>



URL #4

<https://github.com/AICSS442/asa89-IT114003/>



URL
<https://github.com/AICSS442/asa>



Saved: 12/8/2025 1:11:00 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



PT1



PT2



Saved: 12/8/2025 1:13:15 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how important version control is. I was using the wrong version of the template at certain points, which caused a lot of issues that had to be remedied later. Besides that, I learned that methods that send or sync data to clients don't actually perform the sending themselves. Instead, they build a payload with the data and then call `sendToClient()`. That `sendToClient()` method is the place where the actual network communication happens. So on a high level: Each guess is sent as a payload to the server, which processes it and broadcasts updates to all clients. The clients receive these payloads and update their display accordingly, showing the current word, correct guesses, and game messages in real time.



Saved: 12/8/2025 9:41:15 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think the easiest part of the project was probably handling server-side events like letter guesses or user disconnections, because you could process the action and log it with `LoggerUtil.INSTANCE` in one step. For example, after a player guesses a letter, you create the payload, broadcast it to clients, and log the event using something like: `LoggerUtil.INSTANCE.info(String.format("Player %s guessed letter: %s", player.getDisplayName(), letter));`



Saved: 12/8/2025 10:05:28 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think the hardest part was creating and integrating the new payload type for guessed letters and making sure the client properly handled it. It was a little bit of a process because I had to add a new PayloadType, set the guessed letter in the payload on the server, broadcast it, then update the client's processPayload method to handle this new type. I can extend this to creating and integrating new payload types in general as well, that was just a specific example.



Saved: 12/8/2025 10:48:22 PM